Qwen Harness User Manual

Complete guide to the local chat and coding workstation

Application version	1.3.4
User manual	2026

Local AI · Qwen 3.8 · Ornith 1.5 · Windows 11 · RTX 5090

Contents

1. What Qwen Harness Is

Qwen Harness is a private Windows desktop application for running local language models as a general chat assistant, research assistant, writing partner, coding agent, and computer operator. Model inference runs on your workstation through `llama.cpp`; conversations and project files remain on local storage.

The application is designed for one user and one active model. It does not use parallel model agents. The active model can, however, run independent read-only tools in parallel and can leave long commands running in the background.

> NOTE: Model inference is local. When the model uses `web_search` or `web_fetch`, the application makes ordinary internet requests to search engines and public web pages. Disable web access in `config.yaml` or avoid Research/Web requests when you require a fully offline session.

Main capabilities

- General conversation without coding behavior.
- Multi-source web and document research with a final synthesis.
- Writing and revision of scripts, articles, reports, treatments, and other documents.
- Local project development with file, Git, shell, test, image, and rollback tools.
- Screenshot-based control of Windows applications.
- Persistent projects, multiple chats per project, three-layer memory, pinned files, and optional skills.
- English and Czech interface, switchable without losing the active chat.

What the model can see

Selecting a project gives the model access to that directory through tools. It does not place every file body in the context window. The model receives a compact project map and reads individual files when useful. Pinned files and the three memory layers are exceptions: they are deliberately included as persistent context.

2. Requirements and Installation

Recommended workstation

Component	Recommended configuration
Operating system	Windows 11, 64-bit
GPU	NVIDIA RTX 5090 with 32 GB VRAM
Driver	Current NVIDIA driver compatible with the bundled CUDA build
System RAM	Enough for Windows, model mapping, projects, and tools; 64 GB or more is comfortable
Free disk space	At least 65 GiB for all models, runtime, and working data
Python	Python 3.12 when using the source or first-run setup
WebView	Microsoft Edge WebView2, normally present on Windows 11

Other NVIDIA GPUs may work, but the supplied contexts and quantizations were tuned and tested for a 32 GB RTX 5090. Lower-VRAM cards require smaller contexts, lower quantization, fewer GPU layers, or CPU offload.

Installing with Setup.exe

1. Run `QwenHarness-Setup-<version>.exe`.
2. Choose English or Czech in the installer. English is the default.
3. Choose whether to create a desktop shortcut.

4. Keep the post-install environment/model setup selected on the first installation.
5. Allow the console setup to complete. It creates the Python environment, installs dependencies, downloads `llama.cpp`, and downloads the configured models.
6. Start **Qwen3.8-27B Harness** from the Start Menu or desktop.

The default installation directory is:

```
%LOCALAPPDATA%\QwenHarness
```

The complete download is approximately 59 GiB and includes Qwen Q4, Qwen Q5, Ornith Q5, and the vision projectors. Interrupted Hugging Face downloads can normally be resumed by running **Set up environment and models** again from the Start Menu.

First launch

The desktop launcher checks the Python environment, dependencies, `llama.cpp`, and the required model files. Missing components trigger the setup workflow. Once installed, ordinary launches do not download the models again.

Starting the desktop application opens the native WebView window, starts the Web UI, and starts the selected `llama-server` model when required. Closing the desktop application stops its Web UI and model server and releases GPU memory.

Updating

Install the new Setup.exe over the existing installation. User-created data, models, sessions, projects, user skills, and saved UI state are preserved. Changed application files and bundled system skills are updated. Models are downloaded again only when a configured model file is missing.

Uninstalling and removing all data

Windows uninstall removes installed application files. Large runtime data and user-generated data may remain so that an update or reinstall does not require downloading everything again.

To remove everything after uninstalling, inspect and delete the remaining installation directory only when you no longer need:

- `runtime\models` - GGUF model files.
- `sessions` - chat history, attachments, research ledgers, and exports without a project.
- `projects` - projects created inside the application.
- `memory` - global and work-mode memory.
- `user-skills` - personal skills.
- `projects.json` - project registry.

WARNING: Deleting these directories is permanent. Back up projects, sessions, memory, and user skills first.

Running from the source checkout

```
py -3.12 -m venv .venv
.venv\Scripts\python scripts\setup_env.py --model all
.venv\Scripts\python qwen_app.py
```

For terminal use, run `run_cli.bat` or `.venv\Scripts\python tui.py`.

3. Desktop Interface Tour

The application has a scrollable sidebar and the main chat area.

Status panel

The top status panel shows:

- Active or loading model.
- Whether the inference server is running.
- GPU VRAM usage.
- KV cache precision.
- Estimated persistent chat context usage and context limit.
- Live generated-token estimate while the model is thinking, speaking, or preparing a tool call.

Red means the server is down or a switch failed. Green means the selected model is ready. Orange/red context indicators appear near the compression thresholds.

Work mode

The **Work mode** dropdown changes the model's tools and behavior for the active chat. The selected mode is saved with that chat. See the Work Modes chapter for the exact differences.

Server panel

Control	Effect
Start	Loads the selected model and opens the local API server.
Stop	Stops `llama-server` and releases VRAM. Chats remain saved.
Restart	Stops and reloads the selected model, KV profile, and context configuration.

Model and KV changes automatically request a server restart. Loading takes roughly 10-20 seconds depending on the model and disk cache.

Collapsible sidebar groups

Most secondary controls are collapsed to keep the sidebar readable:

- **Context & handoff** - manual compression and a summarized handoff to a new chat.
- **Changes in this task** - files changed since the current task began and one-click rollback.
- **Unfinished task** - resume work saved before an application restart.
- **Long-running operations** - running background commands and their termination control.
- **What the model currently sees** - context statistics and pinned files.
- **Available skills** - skill catalog and personal skill folder.
- **Research progress** - source counts, ledger export, and synthesis export.
- **Settings** - model, KV cache, autonomy, thinking, language, and memory files.

Project and chat area

The project dropdown selects a project or **No project**. Separate lists show chats belonging to the selected project and chats without a project. **Search all chats** searches all persistent chat text.

The highlighted active-chat panel contains:

- **New chat**.
- **Delete** with a second-click confirmation.
- Rename field; type the name and press Enter.
- **Move chat to...** dropdown; selection moves immediately.
- Chat export/import controls.

Main chat and composer

The main area displays the complete user-visible history, including old messages that may already be compressed out of the model's active context.

The composer contains:

- A multiline prompt field.
- **Send** and **Stop**.
- **Attachment** for image files.
- **Retry**, **Undo**, and **Fork** beneath the prompt.

Keyboard behavior:

Key	Result
Enter	Send the prompt.
Ctrl+Enter	Send the prompt.
Shift+Enter	Insert a new line.

The composer clears immediately after sending, even while the model continues working.

4. Models, KV Cache, Context, and Thinking

Installed model profiles

Model	Typical role	Weight quantization	KV choices	Tested context
Qwen 3.8 27B Q5	Main high-quality model	Q5	F16 or Q8	F16 96k; Q8 192k
Qwen 3.8 27B Q4	Faster and largest-context option	Q4	F16 or Q8	F16 128k; Q8 256k
Ornith 1.5 35B-A3B Abliterated Q5	Very fast optional reasoning MoE	Q5	Q8 fixed	Q8 128k

The default new-installation profile is Qwen Q5 with Q8 KV and a 192k context. The application remembers the last selected model and each model's KV choice.

Choosing Qwen Q5 or Q4

Use Qwen Q5 for serious development, architecture, difficult reasoning, and final-quality writing. Use Qwen Q4 when speed or the full 256k context matters more than the small quality advantage of Q5.

Q5 with Q8 is a strong general default. Change to F16 when maximum KV precision matters and 96k tokens are enough.

Ornith

Ornith is a Mixture-of-Experts model and generates very quickly because only part of its weights is active per token. All model weights must still fit in VRAM. In practical application development it may produce weaker results than dense Qwen despite its speed.

Ornith supports thinking on/off natively. The intermediate depths are prompt-guided rather than native `reasoning_effort` values:

- `xhigh` adds deliberate planning, architecture, edge-case, and verification guidance.
- `medium` adds moderate reasoning guidance.
- `low` uses Ornith's ordinary thinking behavior.
- `off` disables thinking.

KV cache precision

KV cache stores the model's attention history. It is separate from the quantization of the model weights.

Setting	Benefit	Tradeoff
F16	Highest KV precision	Approximately half the context of Q8 on the same GPU
Q8	Much larger context with lower VRAM use	Small precision tradeoff in the attention cache

Changing KV precision restarts the server. It does not delete or reset the chat.

Thinking levels

For Qwen, `xhigh`, `medium`, and `low` are sent through the model's native reasoning-effort control. `off` disables the thinking block. The setting can be changed between prompts in the same chat.

Reasoning tokens consume generation time and temporary context while the response is being produced. The UI shows a live estimate. Internal reasoning is not saved in full as persistent visible history; after completion the persistent context grows mainly by the final response and tool messages.

No artificial task or output limit

The production agent has no step limit and no application-level output-token cap. It continues until the task is complete, the model reaches its physical context boundary, a tool/server fails, or the user presses Stop.

5. Sending Prompts, Steering, Stop, and Progress

Ordinary prompt

Write the request in natural language. The model automatically receives the active work mode, project, memory layers, current project snapshot, pinned files, and available skill metadata.

For best results, state the result you want, important constraints, and how completion should be verified. The model is allowed to choose its process unless your prompt specifies it.

Image attachment

Use **Attachment** to add one or more images. Supported formats are BMP, GIF, JPEG, PNG, and WebP. The most recent images remain in active context; older image references stay in history but may be omitted from later API requests.

Images can be photographs, diagrams, screenshots, UI references, error messages, or visual source material. Qwen and Ornith use their matching vision projector automatically.

Steering a running task

Sending another message while the model is working does not merely queue a second independent task. It steers the active one:

1. The clarification is accepted immediately and the composer clears.
2. The current generation stops after the nearest completed sentence or safe chunk.

3. The finished partial response remains visible and saved.
4. The clarification is inserted into the same task.
5. The model resumes with the updated instruction.

Use steering for corrections such as "keep the existing camera behavior", "do not change the API", or "also export this as PDF".

Stop

Stop bypasses the normal Gradio queue. It requests a graceful stop and normally allows the nearest sentence to finish. The finished partial text is retained. A new prompt starts cleanly afterward.

Long-running operating-system processes have their own termination control in **Long-running operations**. Stopping generation does not necessarily stop a background process that was already launched; use the process panel when required.

Live progress

The live chat distinguishes:

- Thinking/reasoning.
- Visible response text.
- Preparation of large tool arguments, such as code for `write_file`.
- Execution of tools, commands, tests, searches, and file operations.
- Seconds without new tokens.

For example, while a large program is being generated the UI can show that content for a named file is growing instead of appearing frozen.

6. Work Modes

Each chat stores its work mode. Changing to another chat restores that chat's mode and project context.

Discussion

Use for ordinary conversation, analysis, learning, brainstorming, planning, and non-programming questions. Discussion does not inject coding procedures. It can use memory, internet, project documents, skills, pinned files, and document export.

Research

Use for web research, literature or market investigation, comparison, and evidence synthesis. Research creates a persistent plan and ledger, keeps all sources regardless of perceived trustworthiness, records contradictions, and produces a final synthesis.

Writing

Use for scripts, articles, reports, treatments, notes, and document revision. Writing can read/write project files, use web and documents, view images, create checkpoints, and export Markdown/DOCX/PDF. It does not expose Git and shell tools unless you switch to Development.

Development

Use for programming and repository work. The model receives an automatic project map and can read/search/write files, apply atomic patches, use Git, run shell commands, start tests, inspect images, maintain a task checkpoint, and roll changes back.

Computer

Includes Development tools plus screenshot, mouse, keyboard, and scrolling controls. Use it when the model must interact with a GUI application rather than only project files.

NOTE: Work modes are capability profiles, not separate model instances. Switching mode does not load another model.

7. Projects and Workspaces

No project

Choose **No project** for conversations that should not access a project folder. Chats without a project appear in their own list. Exports are stored in the chat session directory.

Creating a managed project

1. Open **Project management**.
2. Choose **New**.
3. Enter a project name and confirm.

The application creates a new folder under its `projects` directory, registers it, creates project memory when needed, and uses it as the workspace.

Attaching an existing folder

Choose **Attach**, select an existing directory, and the application registers it as a project without copying its files. The folder itself remains in its original location.

Selecting a project

Select the project in the project dropdown. Relative file paths used by tools resolve inside this workspace. The project chat list updates to show chats whose metadata points to that project.

Multiple chats per project

A project can contain any number of chats. Each chat has independent history, work mode, context compression, pinned files, research ledger, and unfinished-task state. They share the physical project directory and project memory.

Moving a chat

In the active-chat panel, open **Move chat to...** and select a target project or **No project**. The move happens immediately. The chat's workspace, project memory, document library, and model context are reconfigured to the target.

Deleting a project

1. Select the project.
2. Open **Project management**.
3. Click **Delete project + folder**.
4. Read the exact path shown.
5. Click again within eight seconds to confirm.

Deletion removes the project registry entry, every chat belonging to the project, the project folder, and all files inside it.

> WARNING: Attached external folders are also deleted from disk when you confirm project deletion. This is not a detach operation. Critical paths such as the application root, projects root, filesystem root, and home directory are blocked, but you must still verify the displayed path.

If a folder was removed outside the application, the project is marked missing. Reattach or remove the stale project entry as appropriate.

8. Chat Management

New chat

New chat creates a transient empty chat in the current project/mode. It is written to disk only after the first user message.

Delete chat

Click **Delete**, then click again within six seconds. The session folder, messages, attachments, research ledger, task state, and session-local exports are removed. Project files are not affected.

Rename

Type a title in the rename field and press Enter. The chat lists and search index update.

Retry

Retry keeps the last user prompt, removes its answer and any compression state created after it, and asks the model to answer again.

Undo

Undo removes the last complete user/assistant round from the chat. It does not roll back project files. Use **Revert task changes** for filesystem rollback.

Fork

Fork creates a new chat branch ending at the last user prompt. It copies relevant image attachments, project assignment, work mode, and pinned-file list. The original remains unchanged.

Search all chats

Open **Search all chats**, enter a word or phrase, and select a result. Search uses a persistent SQLite full-text index and covers saved chats across projects.

Export and import chat

The active-chat **Export / import** group supports:

- Markdown export for human reading.
- JSONL export preserving the message structure.
- JSONL import as a new chat.

Import never overwrites the original session. Its system prompt is refreshed to the current application version.

Handoff to a new chat

Use **Context & handoff > Hand off** for a long conversation that should continue in a fresh chat. The model summarizes essential goals, decisions, state, findings, and next steps; a new chat is created with that summary while the old history remains intact.

9. Context, Compression, and Pinned Files

Context indicator

The status and **What the model currently sees** panel show estimated tokens, visible versus total messages, images, compression state, and pinned files. The estimate includes the current dynamic project/skill snapshot.

Automatic project snapshot

In Development, the model receives a compact map of the current workspace, file types, key entry points, directories, and top-level Python symbols. The snapshot is added at the current task boundary so changes do not invalidate the reusable prefix of the earlier conversation.

Discussion, Research, and Writing use a project document catalog instead of a coding repository map.

Pinning a file

Open **What the model currently sees** and choose **Pin file**, or tell the model to pin a path. The complete text is refreshed into the context at the start of subsequent tasks in that chat.

Good pinned files include architecture decisions, an active specification, a handoff, or project rules that matter throughout the conversation.

Limits:

- Maximum 10 pinned files.
- Approximately 40,000 characters in total.
- Pinning belongs to one chat; a fork copies the pin list, a separate new chat does not.

Do not pin ordinary source files that the model can read on demand, large logs, or memory documents already injected automatically.

Manual and automatic compression

At approximately 85% of the selected model context, the harness summarizes older messages. The visible chat history is never deleted; only the model's request view changes to system prompt + summary + recent messages.

Use **Context & handoff > Compress** to compress earlier. If the model reports an overflow, the agent performs one automatic compression-and-retry cycle. A second overflow is reported instead of looping forever.

10. Three-Layer Memory

Every task receives up to three complete memory documents:

Layer	Scope	Typical content
Global	Every mode and project	User preferences, universal facts, stable conventions
Work mode	All chats using one mode	Research style, writing preferences, development conventions
Project	One workspace	Decisions, terminology, paths, project-specific facts

Locations:

```
memory\GLOBAL.md
memory\MEMORY.md                (Development mode legacy/default)
memory\modes\discussion.md
memory\modes\research.md
memory\modes\writing.md
memory\modes\computer.md
<project>\QWEN_MEMORY.md
```

Open the active memory files in **Settings > Memory**. You can edit them directly or say, for example:

```
Remember globally that I prefer concise Czech summaries.
Remember for Research that every final answer must preserve contradictory findings.
Remember for this project that the delivery folder is D:\Exports.
```

The model chooses `global`, `mode`, or `project` storage through `save_memory`. Moving a chat to another project immediately changes its project-memory layer.

11. Optional Skills

Skills are reusable `SKILL.md` guidance packages. They do not run a second agent and do not override the user. Only each skill's name and trigger description are included in ordinary context; the body is loaded through `read_skill` when it clearly helps.

Skill precedence

1. Project skill: `<project>\.qwen-skills\<name>\SKILL.md`.
2. User skill: `user-skills\<name>\SKILL.md`.
3. Bundled system skill: `skills\<name>\SKILL.md`.

A higher layer with the same `name` replaces the lower one.

Bundled skills

Skill	Purpose
`systematic-debugging`	Reproduce, trace, test hypotheses, and fix the real cause.
`architecture-options`	Compare meaningful architecture options without overriding requested constraints.
`implementation-verification`	Proportionate requirement, artifact, test, and release verification.
`performance-investigation`	Measurement-first latency, throughput, memory, and bottleneck analysis.
`research-synthesis`	Complete synthesis that retains contradictions and all relevant sources.

Adding a user skill

Open **Available skills > Open skills folder** and create:

```
user-skills\my-skill\SKILL.md
```

Minimum format:

```
---
name: my-skill
description: What this helps with and when the model should load it.
---

# My Skill

Flexible guidance, references, examples, or workflow suggestions.
```

The catalog refreshes automatically. Keep the description specific because it is the model's trigger. Put project-only skills in `.qwen-skills` inside that project.

12. Internet and Research

Ordinary web use

All work modes can use `web_search` and `web_fetch` for current information, public documentation, error messages, and web pages. `web_fetch` handles HTML and can extract text from supported PDF and DOCX downloads.

The default search backend is Google; `config.yaml` can select Google, Bing, or automatic fallback. Fetching is read-only HTTP/HTTPS.

Research workflow

Research mode automatically:

1. Records the question.
2. Creates a persistent research plan before searching.
3. Stores every search query and candidate link.
4. Stores the full readable content of fetched web/local sources.
5. Tracks coverage and source IDs.
6. Keeps contradictory and minority information.
7. Produces a final human-readable synthesis.

Sources are not hidden, discarded, or ranked because the model considers them untrustworthy. The user decides credibility and relevance. The synthesis distinguishes source claims from inference and states uncertainty and missing evidence.

Research panel and exports

The **Research progress** panel shows queries, links, sources read, and status. It can export:

- Complete research ledger containing plans and source material.
- Completed synthesis as DOCX.
- Completed synthesis as PDF.

Asking to export an already completed answer does not start a second research run.

Project documents

Readable project documents include Markdown, text, RST, CSV, JSON, YAML, PDF, and DOCX. Research mode records a local document as a ledger source when it is read.

13. Writing and Document Export

Writing mode keeps the user's intent, voice, structure, and constraints. It can read and revise project files without introducing development terminology unless the user asks for it.

The model can export final text in every work mode using natural requests such as:

```
Save the final answer as PDF.
Export this as a structured DOCX called production-report.
Create a Markdown file with this synthesis.
```

Supported formats:

- Markdown (`.md`).
- Structured Word document (`.docx`).

- PDF with Unicode/Czech text and basic tables/inline formatting.

Output location:

- With a project: `<project>\exports`.
- Without a project: `sessions\<chat-id>\exports`.

Exports produced through project file tools participate in the current task checkpoint and can be reverted when appropriate.

14. Development Workflow

Development mode is intended for complete repository work while keeping the main UI conversational.

Project discovery

At each new task, the model receives a current repository snapshot and can call `repo_overview`, list directories, search text, and read relevant files. It should inspect the actual project before editing.

File operations

- `read_file` reads text with line numbers.
- `search_files` finds case-insensitive text matches with optional filename glob.
- `write_file` creates or fully rewrites a file.
- `apply_patch` performs exact atomic text replacements.
- `view_image` brings an image file into vision context.

Task checkpoint and rollback

The first write in a task starts a change journal. **Changes in this task** lists created/modified files. **Revert task changes** restores all journaled files to their pre-task state, including removing files created by the task.

This rollback is independent of chat Undo. Chat Undo changes conversation history; task rollback changes the filesystem.

Git

The model can inspect status and diff and create a local Git commit. `git_commit` stages the paths explicitly supplied by the model or, when omitted, only files recorded by the current task journal. It never pushes automatically. Ask explicitly when you want a commit or push.

Commands and tests

Short commands run synchronously with a timeout. Long commands run in the background and return a process ID. The model can poll output, send stdin, or terminate the full process tree.

`start_project_check` detects common project checks:

- This harness: `tests/test_core.py`.
- Python: pytest/pyproject.
- Node: `npm test` or `npm run check`.
- Rust: `cargo test`.
- Go: `go test ./...`.

The final verification phase is guidance, not a restriction. Explicit user requirements about architecture, format, scope, or a single-file result take priority.

15. Computer Control

Computer mode adds GUI tools to the complete Development tool set.

Operation cycle

1. The model calls `screenshot`.
2. The screenshot is downscaled if necessary and attached to the conversation.
3. The model identifies controls in image-pixel coordinates.
4. It clicks, moves, scrolls, types, or presses keys.
5. It captures another screenshot to verify the result.

The harness maps screenshot coordinates back to the real display resolution.

Available GUI actions

- Capture primary-monitor screenshot.
- Get screen size and coordinate mapping.
- Move mouse for hover states.
- Left/right/middle click and double-click.
- Scroll up/down at an optional location.
- Type ASCII directly or paste Unicode/long text through the clipboard.
- Press keys and combinations such as Enter, Escape, Ctrl+S, Alt+F4, or Win+D.

Failsafe

Move the physical mouse pointer rapidly to the top-left corner of the primary display to trigger PyAutoGUI's failsafe and interrupt GUI actions.

WARNING: Computer mode can see the primary screen and can perform destructive actions. On-screen text can contain misleading instructions. Use Supervised autonomy for email, payments, account changes, deletion, or any sensitive workflow.

16. Autonomy and Confirmations

Autonomy controls confirmation of WRITE-risk tools. It does not change model intelligence or available context.

Level	Behavior
Supervised	Every write, shell action, and GUI action asks for confirmation. Read-only operations proceed.
Semi	The first WRITE action in a task asks; after approval the rest of that task proceeds.
Auto	WRITE actions proceed without confirmation.

There is no task-step limit in any production autonomy mode.

When confirmation is required, the chat lists the pending actions and shows **Allow** and **Deny**. Denying returns the decision to the model so it can adapt. Repeated clicks are ignored once the pending action is gone.

Read-only commands are classified conservatively. Redirection, package installation, arbitrary Python, copying, deletion, network writes, Git commit/push, and mixed command chains are treated as writes.

17. Terminal UI and CLI

The installer creates **Qwen3.8-27B Harness (CLI)** in the Start Menu. It opens `run_cli.bat`, which launches the interactive terminal UI using the installed Python environment.

Commands

Command	Purpose
`/memory`	Show global, active-mode, and project memory.
`/model q4|q5|ornith_q5`	Switch model; restarts the server.
`/work discussion|research|writing|development|computer`	Select work mode.
`/mode chat|agent|computer`	Compatibility shortcut for legacy agent mode.
`/autonomy supervised|semi|auto`	Set confirmation behavior.
`/thinking xhigh|medium|low|off`	Set reasoning depth.
`/ws [path]`	Show or set workspace.
`/img <path>`	Attach an image to the next prompt.
`/screenshot`	Capture and attach the screen.
`/new`	Create a new session.
`/sessions`	List sessions.
`/load <id>`	Load a session.
`/server status|start|stop`	Control the inference server.
`/help`	Show command help.
`/exit`	Exit the CLI.

During confirmation, y allows, n denies, and a allows all remaining writes in the current task. Ctrl+C interrupts generation.

18. Files, Storage, Backup, and Logs

Important locations

Path	Contents
`runtime\models`	GGUF models and vision projectors
`runtime\llama`	`llama.cpp` CUDA binaries
`runtime\webui-state.json`	Last model, KV choice, language, mode, workspace, and active session
`sessions\<id>`	Messages, metadata, attachments, task state, research, compression, exports
`projects`	Managed project folders created by the app
`projects.json`	Registered project list
`memory`	Global and work-mode memory
`<project>\QWEN_MEMORY.md`	Project memory
`skills`	Bundled system skills
`user-skills`	Personal skills preserved by the installer
`<project>\.qwen-skills`	Project skills

Backup recommendations

Back up at least:

- Project directories.
- `sessions` when chat/research history matters.
- `memory`.
- `user-skills`.
- `projects.json`.

Model files can be downloaded again and usually do not need backup.

Logs

Log	Use
`runtime\launcher.log`	Desktop launcher and startup problems
`runtime\app.log`	Native app lifecycle/crash details
`runtime\webapp.log`	Web UI startup and Python errors
`runtime\llama-server.log`	Model loading, context, CUDA, inference, and timing

19. Troubleshooting

Server does not start

- Open the Server panel and try Restart.
- Confirm the selected model exists in `runtime\models`.
- Read `runtime\llama-server.log`.
- Check NVIDIA driver/GPU availability.
- Stop another process using port 8080.
- If VRAM allocation fails, close other GPU applications, select a smaller model/context, or use the tested profile.

Web UI port is occupied

The launcher searches for an available nearby Web UI port. If another Qwen Harness Web UI already owns the configured port, the launcher reuses it. See `runtime\launcher.log` for the selected URL.

Model or projector is missing

Run **Set up environment and models** from the Start Menu. Existing complete files are skipped. The downloader resumes supported incomplete downloads.

Model is slow after a long conversation

- Check context usage.
- Keep Qwen Q5/Q8 for 192k or switch Q4/Q8 for 256k.
- Use manual compression or Hand off.
- Avoid repeatedly changing early persistent memory during one long task because a changed system prefix must be evaluated again.

Context overflow

The agent automatically compresses and retries once. If it overflows again, manually compress, hand off to a fresh chat, choose a larger context profile, or remove unnecessary pinned files/images.

Interface appears frozen

Read the live activity line. The model may be thinking, generating a large tool call, writing a file, running a test, or waiting for a long process. If there is no server activity for an extended period, the harness detects a stale stream and reports it. Use Stop when appropriate.

Stop did not end an operating-system process

Stop ends model generation. Open **Long-running operations** and terminate the process separately.

A task was interrupted by restart

Open **Unfinished task** and choose **Continue task**. Pending confirmations, partial visible text, journal state, and persistent process metadata are restored when possible.

Project folder is missing

The project list marks missing directories. Reattach the correct folder, move affected chats, or remove the stale project. Be careful: confirmed project deletion removes the folder and chats rather than merely unregistering it.

Where is an exported PDF or DOCX?

- Project active: `<project>\exports`.
- No project: `sessions\<chat-id>\exports`.
- Research panel synthesis export: the selected file is also presented by the UI.

Language did not fully change

Choose **Settings > Language**. The interface reloads and preserves the current session. If a native launcher message remains in the old language, restart the desktop application. The saved UI choice takes precedence over the installer-language file.

20. User-Facing Tool Reference

You normally request these operations in natural language; the names below explain what the model can call.

Available in every work mode

Tool	Capability
`read_memory`	Read one complete memory layer.
`save_memory`	Save a durable fact to global, mode, or project memory.
`web_search`	Search current public web results.
`web_fetch`	Read a public HTTP/HTTPS page or supported document.
`context_status`	Inspect tokens, messages, images, pins, and compression.
`pin_context_file`	Pin a text file to the current chat.
`unpin_context_file`	Remove one pin.
`list_project_documents`	List readable documents in the project.
`read_project_document`	Read text, Markdown, PDF, DOCX, JSON, YAML, or CSV.
`list_skills`	List optional skill metadata.
`read_skill`	Load one selected `SKILL.md`.
`export_document`	Export Markdown, DOCX, or PDF.

Added in Writing, Development, and Computer

Tool	Capability
`list_dir`	List folders/files and sizes.
`read_file`	Read text with line numbers and optional range.
`write_file`	Create or overwrite a complete file.
`apply_patch`	Apply exact atomic replacements.
`list_task_changes`	List journaled files for the task.
`undo_task_changes`	Restore every journaled task change.
`search_files`	Search text recursively with an optional glob.
`view_image`	Attach a local image for visual analysis.

Added in Development and Computer

Tool	Capability
`repo_overview`	Return the automatic repository map.
`start_project_check`	Detect and start the primary test/check command.
`git_status`	Show branch and file status.
`git_diff`	Show working or staged diff.
`git_commit`	Create a local commit from selected/current-task files.
`run_command`	Run a short Bash, PowerShell, or cmd command.
`start_command`	Start a persistent background command.
`poll_command`	Read incremental process output/status.
`send_stdin`	Send input to a background process.
`terminate_command`	Terminate a process and its child tree.

Added in Computer

Tool	Capability
`screenshot`	Capture and attach the primary display.
`get_screen_info`	Read screen and image coordinate dimensions.
`move_mouse`	Move pointer to image coordinates.
`click`	Click or double-click with a selected mouse button.
`scroll`	Scroll at an optional image position.
`type_text`	Type or paste Unicode text.
`press_key`	Press a key or key combination.

21. Practical Request Examples

```
Discussion: Compare these two production approaches without introducing coding procedures.

Research: Research the current state of virtual production stages. Keep contradictory findings,
show uncertainty, and export the final synthesis as PDF.

Writing: Read the project treatment, rewrite act two in the same voice, and save a DOCX.

Development: Inspect this repository, reproduce the reported failure, fix the root cause,
run the relevant tests, and summarize what changed.

Memory: Remember for this project that final renders go to D:\Show\Delivery.

Pinning: Pin docs\ARCHITECTURE.md for this chat.

Computer: Take a screenshot, open the application settings, enable the requested option,
and verify the result with another screenshot.

Export: Save your existing answer as PDF. Do not research again.
```

22. Operational Notes and Limitations

- One large model occupies nearly the full GPU, so model inference is intentionally single-slot.
- Read-only tools and background processes can still run concurrently.
- Switching model or KV cache restarts the server and clears its transient prompt cache, but not chat history.
- Context numbers are estimates; exact tokenization and image cost depend on the model/template.
- Model output can be wrong. Verification tools improve confidence but do not guarantee correctness.
- Computer control is limited to the primary monitor and depends on visible UI state.
- Internet extraction may fail on login walls, scripts-only sites, CAPTCHAs, blocked downloads, or unsupported formats.
- Public skills should be reviewed before installation. Skill text guides the model and can influence its actions.
- Git commit is local unless the user explicitly asks for a push and the environment permits it.
- The application is a personal local tool. Maintain backups of irreplaceable project and session data.